Neuro-Symbolic Agentic Code Intelligence: A Rule-Governed NSN Pipeline for Agent Suites and SWE-bench Patch Generation

Adithya Kishor
Extra Project
Roll Num: 2023111019
Project repository: https://gitfront.io/r/afoolishman/rPbsa8yXe8Tm/se-symbolic-ai-work/

***Abstract*—Autonomous LLM agents are increasingly used for software engineering tasks such as support, research, and patch generation, but purely neural pipelines remain brittle under safety, format, and policy constraints. This project implements and evaluates a lightweight Neuro-Symbolic-Neuro (NSN) architecture that wraps LLM generations with deterministic symbolic checks and iterative refinement. We contribute (i) an *agentic system* for defining YAML-configured agents with rule sets, tool access, memory bounds, and test suites that compare NSN to neural-only baselines; (ii) a *SWE-bench slice runner* that generates unified diffs under a static-analysis rule engine and exports predictions for SWE-bench harnesses; and (iii) an analysis of how symbolic governance improves output quality (well-formed diffs, citation presence, safety conformance) while trading off latency. We report results on a curated 10-instance SWE-bench Lite slice and a rule-driven agent test suite, and outline planned extensions including stronger program analysis, execution-based verification, and learning-to-compile symbolic constraints from failures.**

***Index Terms*—neuro-symbolic AI, agentic systems, software engineering, SWE-bench, LLM governance, symbolic rules, iterative refinement**

I. Introduction

LLM-based assistants are shifting toward *agentic* systems: multi-turn entities that plan, call tools, and produce artifacts such as patches. Despite strong generative capacity, neural-only agents can violate non-negotiable constraints (e.g., PII policies, tool-use restrictions, citation requirements) and can fail to produce machine-consumable artifacts (e.g., malformed diffs). Motivated by architectural patterns for neuro-symbolic multi-agent systems, the project aims to make agent outputs more *deterministic, auditable, and constraint-aligned* by integrating symbolic rule engines directly into the generation loop.

This work operationalizes a pragmatic NSN pattern: *Neural generation* produces an answer or patch; *Symbolic checking* deterministically validates it against explicit rules; *Neural refinement* rewrites the output using violation feedback, repeated for a bounded number of rounds.

II. Background and Motivation

The accompanying project write-up emphasizes that purely neural multi-agent systems lack deterministic reliability and explainability for mission-critical settings, motivating neuro-symbolic integration [**?**]. In software engineering, this gap is visible in: (i) safety policy adherence (PII leakage, unsafe instructions), (ii) formatting and protocol compliance (diff structure, required citations), and (iii) predictable behavior under governance (blocking, redaction, constrained autonomy).

III. Related Work (Brief)

Benchmarking LLMs for software engineering has been popularized by SWE-bench, which evaluates whether models can resolve real GitHub issues by producing patches that pass repository tests [**?**]. In parallel, multiple lines of work explore *agentic* workflows (planning, tool use, multi-step refinement) and *guardrails* (policy enforcement, structured outputs). This project focuses on an engineering-centric slice of that space: a minimal symbolic layer that is easy to implement, fast to extend, and produces human-auditable violation reports, while remaining compatible with standard patch-based harnesses.

IV. System Overview

A. Architectural Patterns Adopted

Guided by neuro-symbolic multi-agent architecture patterns, the implemented design instantiates the following reusable patterns:

- **Central orchestration with constrained autonomy**: a CLI runtime mediates model calls and enforces pre/post checks; agents do not directly "self-execute" actions outside declared tools.
- **Hierarchical memory discipline**: conversational memory is bounded (turn window) and treated separately from explicit symbolic state (rule definitions, test assertions, violation logs).
- **Deterministic governance layer**: symbolic checks provide hard constraints and structured violation reports that remain stable across runs, enabling regression testing of safety and formatting behavior.
- **Neuro-Symbolic-Neuro refinement**: violations are fed back as machine-generated critiques that drive targeted rewriting.

B. Agentic System (Rule-Governed Agents)

The `agentic_system/` component provides a CLI to *define*, *inspect*, *chat*, and *test* agents. Agents are configured via YAML with:

- **Role prompt**: long-form task specification.
- **Model and decoding**: OpenRouter-hosted chat model and max tokens.
- **Bounded memory**: rolling window of prior turns.
- **Symbolic rule set**: built-in rules (e.g., `no_pii`, `no_code_execution`, `must_cite_sources`, `no_bare_except`) and custom regex rules.
- **Tools**: optional deterministic utilities (e.g., word count, uppercase) exposed as tool calls.

C. NSN Runtime

For each user turn, the runtime executes:

1) **Pre-check (input)**: symbolic validation; optionally block if violations occur.
2) **Neural generation**: single LLM call, with rule constraints injected in the system prompt (NSN mode) or without them (neural-only baseline).
3) **Post-check (output)**: symbolic validation of the generated output.
4) **Refinement loop**: if violations exist and refinement budget remains, the LLM is called again with structured violation feedback to produce a corrected output.

This design yields a *bounded* and *auditable* control loop: violations are explicit, checks are deterministic, and refinement rounds are capped.

D. Symbolic Contract and Refinement Prompting

The symbolic layer defines an explicit contract C over an output y (answer text or patch). Each rule $r_i \in C$ deterministically maps $y \mapsto \{\text{pass}, \text{fail}\}$ with an explanation string. In NSN mode, violations are converted into a structured refinement request:

- **Constraints as instructions**: a human-readable list of active rule descriptions is injected before generation.
- **Violations as feedback**: the post-check produces a list of failures; the refinement prompt includes the current output and enumerated violations to fix.

This makes refinement *targeted* (fix specific failures) rather than generic self-critique.

E. SWE-bench Patch Generation Runner

The `swe_bench/` component benchmarks NSN versus neural-only on a curated 10-instance SWE-bench Lite slice. Each task prompts the model to output *only* a unified diff. A symbolic rule engine performs deterministic static checks over *added lines* and diff structure. Active checks include:

- no shell-injection primitives (`os.system`, `eval`, `exec`);
- no hardcoded secrets (`api_key`, `token`, etc.);
- no bare `except:`;
- patch shape requirements (presence of `---`/`+++` headers and `@@` hunks);
- no debug prints on added lines.

NSN adds constraint text to the prompt and refines a patch up to two times if violations are detected; neural-only performs no refinement.

V. Evaluation Setup

A. Metrics

We report:

- **Patch validity** (`patch_valid`): whether the output is a non-trivial unified diff containing `---`/`+++` file headers and at least one `@@ ... @@` hunk header. This is a *harness-compatibility proxy*: invalid patches cannot be applied or evaluated by downstream tooling.
- **Symbolic safety rate** (`safety_rate`): fraction of active symbolic checks passed by the *final* output:

$$\texttt{safety\_rate} = \frac{\#\texttt{checks\_passed}}{\#\texttt{checks\_total}}.$$

  In `swe_bench/`, checks are patch-focused safety/shape constraints; in `agentic_system/`, checks correspond to the enabled agent rule set (e.g., no-PII, must-cite-sources).
- **Refinement rounds** (`refinement_rounds`): number of correction iterations executed by NSN after post-check failures, capped by configuration (≤ 2 in `swe_bench/`; agent YAML config controls this in `agentic_system/`).

B. Experimental Protocol: SWE-bench Slice Runner

The SWE-bench experiment uses the offline 10-instance slice in `swe_bench/swebench_sample.py`. For each instance and approach (`neural_only`, `nsn`):

- The model receives a system instruction to output *only* a unified diff in a ```` ```diff ```` fence.
- In NSN, a human-readable list of symbolic constraints is appended to the user prompt (e.g., "Patch must contain `@@` hunk headers").
- The patch is extracted from the first fenced diff/patch block; if none exists, raw output is used.
- Symbolic checks are applied to the patch text (Section VI-A).
- In NSN, if *any* violations exist, a refinement prompt is constructed containing: (i) the current diff, and (ii) the enumerated violation messages; the model must return a corrected diff. This repeats until violations disappear or refinement budget is exhausted.

C. Experimental Protocol: Agentic Rule Tests

The agentic experiment uses YAML suites (e.g., `agentic_system/tests/`) where each case defines an input and assertions. For each case and approach:

- The agent memory is cleared per-approach per-case (clean A/B comparison).
- **Pre-check**: input is validated; if blocking rules trigger and `block_on_input_violation=true`, the turn is blocked.
- **Generation**: the LLM produces an output (NSN includes constraints; neural-only does not).
- **Post-check and refinement**: output is validated; if output violations occur and refinement is enabled, the runtime asks the model to rewrite with violation feedback.

TABLE I
ACTIVE SYMBOLIC CHECKS IN SWE_BENCH/ (STATIC ON PATCH TEXT).

Rule	Meaning (what it prevents)
`diff_has_file_header`	Patch must contain `--` and `+++` file markers; prevents non-unified output.
`diff_has_hunk_header`	Patch must contain at least one `@@ ... @@`; prevents empty or context-less diffs.
`no_bare_except`	No added line may introduce `except:`; prevents overly broad exception handling.
`no_shell_injection`	Forbids `os.system(`, `eval(`, `exec(` on added lines; prevents obvious injection primitives.
`no_hardcoded_secrets`	Forbids patterns like `api_key = "..."`; prevents accidental secret materialization.
`no_debug_prints`	Blocks added debug prints matching "print(...debug...)"; reduces low-quality patches.

TABLE II
SWE-BENCH SLICE RESULTS (10 TASKS).

Approach	Patch-valid	Avg. safety rate	Refinements
Neural-only	0.90	0.967	0
NSN	1.00	1.000	≤ 2 (bounded)

- The test runner then evaluates assertions: text contains/does-not-contain, blocked/not-blocked, and whether symbolic pass/fail matches expectation.

D. Agent Test Suites

Test suites are written in YAML and executed by a runner that asserts: **content constraints** (must contain / must not contain phrases), **blocking behavior**, and **symbolic pass/fail** (whether post-violations should be empty).

E. SWE-bench Slice

We use a fixed offline 10-task slice representing diverse repos (e.g., Django, Matplotlib, Scikit-learn, SymPy). The evaluation in this project focuses on **artifact validity** (unified diff structure) and **symbolic safety compliance** for patch text. Full SWE-bench dockerized execution is a planned next step (Section VII).

VI. RESULTS AND ANALYSIS

A. Symbolic Rule Sets Used in Experiments

Table I lists the active SWE-bench symbolic checks. These are intentionally lightweight and deterministic, designed to prevent common "pipeline-breakers" (malformed diffs) and obviously unsafe additions (secrets, shell injection).

B. SWE-bench Slice: Patch Validity and Symbolic Safety

Table II summarizes outcomes. Neural-only produced one empty/malformed patch (missing diff headers/hunks), while NSN produced well-formed diffs across all 10 tasks.

Observed quality improvement. The symbolic checks act as an output contract: the diff must be machine-consumable and must not introduce flagged patterns. The NSN loop converts this contract into actionable feedback, enabling recovery from malformed outputs without manual intervention.

TABLE III
PER-DIFFICULTY CONFORMANCE ON THE 10-TASK SWE-BENCH SLICE.

Difficulty	Patch-valid		Avg. safety	
	Neural	NSN	Neural	NSN
Easy (4)	0.75	1.00	0.917	1.000
Medium (5)	1.00	1.00	1.000	1.000
Hard (1)	1.00	1.00	1.000	1.000

Failure recovery example. In the Django task `django__django-11179`, the neural-only approach returned an empty patch, failing both diff-shape rules. NSN performed two refinement rounds and produced a unified diff with valid headers and hunks, satisfying all symbolic checks. While this does not guarantee semantic correctness, it meaningfully reduces "pipeline breaks" where downstream harnesses cannot even evaluate a submission.

What the numbers mean.

- **Patch-valid 0.90 vs 1.00**: in 1/10 tasks, the neural-only run produced an artifact that could not be treated as a unified diff. NSN eliminated this failure mode on the slice.
- **Safety 0.967 vs 1.000**: neural-only failed, on average, 3.3% of enabled checks (often driven by empty/malformed diff artifacts which fail structural checks). NSN satisfied all checks for all tasks in the slice.

Trade-off. NSN increases latency due to extra symbolic passes and occasional additional LLM calls. In this implementation, latency is the wall-clock duration of the end-to-end generation (including refinement). This overhead is justified when the system value is in *automation reliability* (fewer manual retries, fewer harness failures).

C. SWE-bench Slice: Per-Difficulty Breakdown

Table III shows patch-validity and symbolic compliance stratified by the slice difficulty labels. The key observation is that symbolic conformance remains stable across difficulty for NSN, whereas neural-only can fail even on an "easy" instance if it returns an empty or malformed artifact.

D. Agentic System: Rule-Constrained Output Quality

From a sample report (`agentic_system/report.json`) with 5 cases per approach:

- **Pass rate**: both approaches achieved 4/5 on the suite, but for different reasons (baseline failed a citation/language requirement case; NSN failed an expectation where the suite anticipated a violation that did not occur).
- **Safety alignment**: NSN maintains a deterministic post-check and can iteratively enforce constraints such as mandatory citations.
- **Behavioral governance**: input blocking (e.g., explicit code execution requests) is enforced pre-generation, preventing unsafe interactions.

TABLE IV
AGENTIC SUITE SUMMARY (5 CASES PER APPROACH).

Approach	Suite pass rate	Avg. safety rate	Avg. refinements
Neural-only	0.80	0.80	0
NSN	0.80	1.00	0.8

E. Agentic Test Suite Summary

Table IV aggregates the same report into suite-level metrics. Although pass rate is similar in this small sample, NSN provides higher controllability when requirements are strict (e.g., citations present, language constraints) because violations can be surfaced and repaired automatically.

Interpreting these numbers.

- **Suite pass rate**: proportion of test cases whose *assertions* match the observed behavior. This is a functional metric over the test harness, not a direct measure of correctness in the open world.
- **Avg. safety rate**: fraction of symbolic constraints satisfied by the final output. Here, the NSN safety rate is higher because it explicitly performs post-check and can refine to satisfy requirements (e.g., "must include a citation marker").
- **Avg. refinements 0.8**: on average, NSN needed just under one correction call per test case. This indicates that the symbolic layer is not only detecting problems but also enabling automatic repair.

F. Notes on How Symbolic AI Improved Output Quality

Across both components, the symbolic layer improved output quality via:

- **Format reliability**: diff-shape rules prevent empty/unparseable patches and enable downstream automation (harness ingestion, patch application).
- **Safety and policy compliance**: PII, code-execution requests, and other forbidden patterns are caught deterministically.
- **Explainability**: each violation is attributed to a named rule with a human-readable explanation, supporting audits and debugging.
- **Bounded correction**: refinement rounds provide a controlled repair mechanism instead of open-ended agent behavior.

G. Limitations

The current results primarily quantify *conformance* (valid diff shape and rule compliance), not *functional correctness*. The symbolic rules are intentionally lightweight (regex and structural checks) and may miss deeper issues (e.g., subtle security vulnerabilities, semantic regressions, incomplete fixes). Additionally, refinement can sometimes "overfit" constraints by producing superficially compliant but low-quality outputs; mitigating this requires stronger objective signals (tests, static analysis, or reward models).

VII. PLANNED FUTURE WORK

This project is intentionally modular to enable progressively stronger symbolic governance and stronger software-engineering evaluation:

- **Execution-based verification**: run generated patches in containerized SWE-bench evaluation to measure task resolution, not only diff validity.
- **Stronger static analysis**: integrate language-aware linters/AST checks (e.g., Python AST safety rules) beyond regex matching; add repository-specific constraints.
- **Constraint learning**: automatically propose new symbolic rules from recurring post-violations and failure clusters (meta-governance).
- **Multi-agent orchestration**: extend from single-agent NSN loops to multi-agent collaboration with centralized symbolic policy enforcement and shared symbolic memory.
- **Cost/latency optimization**: adaptive refinement (stop early on high-confidence passes), caching of check results, and selective rule activation per task type.
- **SWE-bench integration details**: integrate repository checkout and patch application to run unit tests locally, enabling per-task correctness diagnostics and richer error-aware refinement (e.g., failing test traces as symbolic signals).

A. Extra Work Planned (Expanded)

The next planned milestones focus on moving from *conformance* to *correctness* while keeping symbolic governance first-class:

- **Patch application and rollback**: automatically apply diffs to a clean checkout and validate they apply cleanly; if not, refine using the apply error as a symbolic signal.
- **Test-driven refinement**: incorporate failing unit test output into the refinement prompt while keeping safety/format rules active.
- **Repository-aware constraints**: enable per-repository rule profiles (e.g., "no new dependencies"; "touch only specified files"; "preserve public API").
- **Long-horizon agent workflows**: add multi-step planners (issue triage $\rightarrow$ hypothesis $\rightarrow$ patch $\rightarrow$ test) governed by the same rule engine at each artifact boundary.
- **Evaluation at scale**: run larger SWE-bench slices and compare not just patch-validity but task resolution rate, with ablations over rule sets and refinement budgets.

VIII. CONCLUSION

We implemented a practical NSN neuro-symbolic pattern for software engineering agents and patch generation. Results on a curated SWE-bench Lite slice show improved artifact validity and perfect symbolic safety compliance under explicit rules, demonstrating that symbolic governance can systematically improve output quality and auditability in agentic pipelines. The next phase is to couple these guarantees with execution-based correctness to quantify end-to-end gains on SWE-bench.

ARTIFACTS

Project components referenced in this draft: `agentic_system/` (rule-governed agent framework), and `swe_bench/` (SWE-bench slice runner and outputs).